Babel Costa Rica · Objetivos de desarrollo 2026 · Evidencia de cumplimiento

Kevin Solano Muñoz
Software Engineer FullStack
Fecha: 28 de setiembre de 2026

Desarrollo web moderno y programación asistida por IA

React 19 + TypeScript · .NET 10 · Claude Code · Clean Architecture

Estado: 100% completado

Objetivo. Desarrollar competencias en desarrollo web moderno y programación asistida por Inteligencia Artificial mediante la construcción de un módulo de gestión de gastos personales utilizando React y TypeScript, complementado con formación práctica en Claude Code y su aplicación a lo largo del ciclo de desarrollo.

Alineación organizacional: Adoptar el uso de la IA Generativa en el día a día de todos los que formamos parte de Babel Costa Rica. · **Repositorio:** github.com/kevinsonalo/gastos-app

1. Resumen ejecutivo

Se construyó **"Mis Gastos"**, un módulo de gestión de gastos personales con frontend en **React 19 + TypeScript** y un backend opcional en **ASP.NET Core 10** con base de datos SQLite. Claude Code se usó como asistente en **21 actividades** del ciclo de desarrollo (arquitectura, diseño, implementación, pruebas, revisión de código, refactorización, documentación y diagnóstico de entorno). Cada resultado se validó con compilación estricta, pruebas automatizadas, linter y revisión manual antes de aceptarlo.

4 / 4	21	125	11	19
Key Results completados	actividades con Claude documentadas (meta: 10)	pruebas automatizadas (77 front · 48 back)	decisiones técnicas documentadas (ADR)	commits por hito + CI en GitHub Actions

2. Key Results y evidencias

Key Result	Evidencia entregada	Estado
KR1. Diseñar la arquitectura del módulo con Claude Code como asistente técnico	Diagrama de componentes y vista del sistema, modelo de datos (ERD) y 11 ADRs en `docs/ARQUITECTURA.md`. Guía de estructura y patrones en `docs/GUIA_BUENAS_PRACTICAS.md`.	100%
KR2. MVP funcional: registro de gastos, categorías, dashboard básico y persistencia	Aplicación funcionando (capturas en la sección 4). Persistencia en el navegador o en API REST .NET + SQLite. Repositorio Git con 19 commits por hito e integración continua.	100%
KR3. Aplicar Claude Code en al menos 10 actividades distintas del ciclo de desarrollo	21 actividades en `docs/BITACORA_PROMPTS.md` (prompt, resultado, validación, aprendizaje); 23 hallazgos en `docs/CODE_REVIEW.md`; `CLAUDE.md` generado con `/init`.	100%
KR4. Completar el curso de Claude Code y documentar los aprendizajes	Curso completado. Aprendizajes y buenas prácticas de prompts en `docs/MEJORES_PRACTICAS_PROMPTS.md`.	100%

3. KR1 — Arquitectura

El frontend y el backend siguen la misma arquitectura (**Clean Architecture**): las dependencias apuntan hacia el dominio y la infraestructura implementa interfaces (puertos) definidas por la aplicación. Cada proyecto tiene una **prueba automática** que falla si se rompe la regla de dependencias.



Patrones de diseño aplicados

Patrón	Para qué	Dónde
Clean Architecture / Ports & Adapters	Aislar el negocio de React, del navegador y de la base de datos	Ambos proyectos
Repository + Unit of Work / Gateway	Cambiar la persistencia (navegador ↔ API) sin tocar el negocio	`StoreGateway`, `IExpenseRepository`
CQRS con handlers propios	Un caso de uso por clase, sin dependencias de terceros (MediatR pasó a licencia comercial)	`CreateExpenseHandler`…
Result pattern + Problem Details (RFC 9457)	Errores de negocio como valores, con mensajes por campo iguales en front y back	`Result<T>`, `Decision`
Reducer, Custom Hooks, actualización optimista	Estado predecible y UI que responde al instante; se revierte si el servidor rechaza	`useExpenseStore`, `useExpenseForm`
Inyección de dependencias (reloj e ids)	Pruebas deterministas sin mocks globales	`Clock`, `IClock`

Decisiones destacadas: ADR-005 (fechas locales `YYYY-MM-DD` para evitar el desfase UTC-6), ADR-008 (no se elimina una categoría con gastos, validado en front, API y base de datos), ADR-011 (backend opcional y puerto de persistencia asíncrono).

4. KR2 — MVP funcional

- **Registro de gastos:** monto, descripción, categoría, fecha y método de pago (Tarjeta, Efectivo, SINPE Móvil, Transferencia). Alta, edición y eliminación con validación.
- **Categorías:** 8 por defecto; crear, renombrar, color; eliminación bloqueada si tiene gastos.
- **Dashboard:** total del mes y variación, promedio diario, categoría principal, gasto por categoría y tendencia de 6 meses.
- **Persistencia:** `localStorage` o API REST .NET 10 + SQLite (6 meses de datos de prueba).
- **Respaldo:** exportar e importar JSON validado.
- **Calidad:** TypeScript y C# estrictos, 125 pruebas, pruebas de arquitectura y CI en GitHub Actions para ambos proyectos.

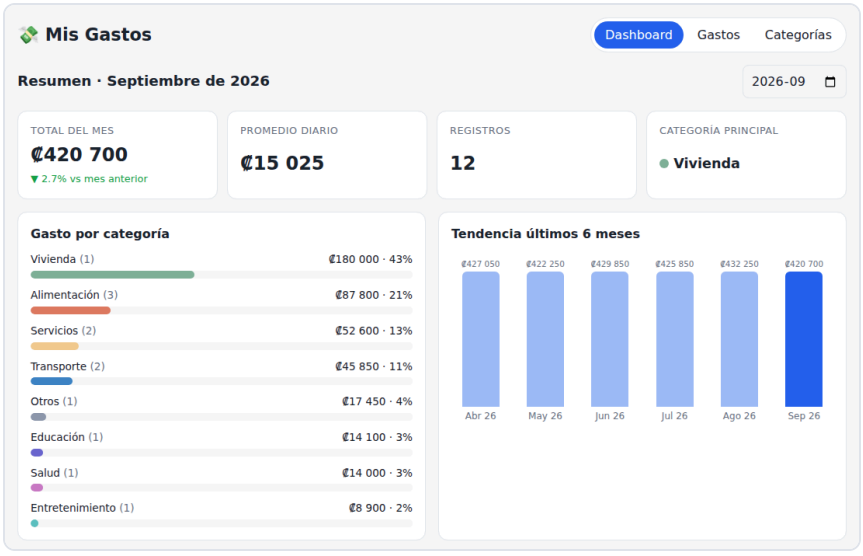


Figura 1. Dashboard con datos servidos por la API .NET (pie de página: "Datos guardados en la API").

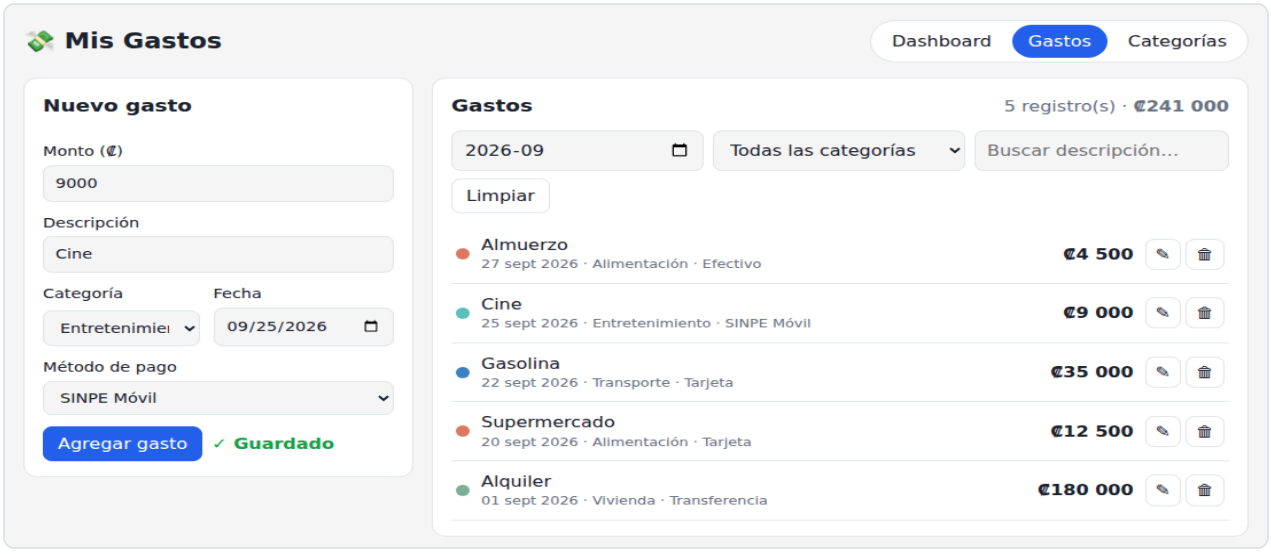


Figura 2. Registro y lista de gastos con filtros por mes, categoría y texto.

5. KR3 — Claude Code en el ciclo de desarrollo

Las 10 actividades de ejemplo del Key Result se cubrieron todas; en total quedaron documentadas 21 actividades en la bitácora.

Actividad (KR3)	Uso de Claude Code	Evidencia / validación
Definición de arquitectura	Propuesta de alternativas con trade-offs; decisión propia (localStorage primero, API después)	ADR-001, ADR-009, ADR-011
Diseño de componentes	Separación de lógica y vista: `useExpenseForm`, `Field`, textos fuera del dominio	ADR-010 · lint 0 warnings
Revisión de código	3 rondas de revisión: 23 hallazgos, 19 aplicados y 4 diferidos con justificación	`CODE_REVIEW.md`
Refactorización	Migración a Clean Architecture con Claude Code en la terminal de VS Code	Commit `main update structure`
Generación de pruebas	Casos borde: año bisiesto, `0.1 + 0.2`, cambio de año, JSON corrupto, reglas de arquitectura	77 Vitest + 48 xUnit
Documentación técnica	`/init` → `CLAUDE.md`; README, guía de buenas prácticas, ficha del proyecto	Carpeta `docs/`
Optimización de consultas	Índices por fecha y categoría; filtros en SQL y búsqueda sin distinción de tildes en memoria; totales en memoria por limitación de SQLite con `decimal`	`ExpenseRepository`, ADR-011
Detección de errores	Desfase de zona horaria, error 500 por enum inválido, ids demasiado largos, validación incompleta	Corregidos con prueba
Diseño de APIs	11 endpoints REST, Problem Details, CORS, archivo `.http` de pruebas manuales	`api/README.md`
Elaboración de roadmap	Fase 2: migraciones EF, SQL Server/Azure SQL, Entra ID, presupuestos	ARQUITECTURA §7
Adicional: diagnóstico de entorno	Node 18 vs Vite 8, credenciales Git de otra cuenta, feed NuGet privado vencido	Resueltos a nivel repositorio

6. KR4 — Curso y aprendizajes

Buenas prácticas adoptadas

- **Contexto primero:** `CLAUDE.md` con convenciones y comandos del proyecto.
- **Plan antes que código** en cambios de varios archivos.
- **Pedir opciones con trade-offs**: la decisión final es del desarrollador.
- **Tareas pequeñas y verificables**, un commit por hito.
- **Cerrar el ciclo con verificación automática** (tests, build, lint, CI).

Uso responsable de la IA

- Todo cambio se leyó, se entendió y se validó antes de aceptarlo.
- Se descartaron sugerencias sobrediseñadas (p. ej. virtualizar listas).
- Sin secretos ni datos de clientes en los prompts.
- Permisos en modo "pedir aprobación".
- Uso de IA declarado en la documentación del proyecto.

7. Conclusión

El objetivo se cumplió en su totalidad y superó las metas planteadas: 21 actividades con Claude (meta: 10) y un backend .NET adicional que demuestra la arquitectura de punta a punta. El principal aprendizaje es que la IA acelera el trabajo cuando se le da contexto y restricciones claras, y que **la verificación automática y el criterio técnico propio son los que garantizan la calidad** del resultado.

Documentos en el repositorio (github.com/kevinsonalo/gastos-app): `README.md` · `docs/PROYECTO.md` · `docs/ARQUITECTURA.md` · `docs/GUIA_BUENAS_PRACTICAS.md` · `docs/BITACORA_PROMPTS.md` · `docs/CODE_REVIEW.md` · `docs/MEJORES_PRACTICAS_PROMPTS.md` · `api/README.md` · `CLAUDE.md`